

Response to Updated Review C

Thank you for the careful follow-up. We revised the paper to address both issues: the scope of the impossibility theorem and the unmeasured performance of IntConv detection.

1. Scope and significance of the impossibility result

We agree that the previous version did not state an important limitation clearly enough. The theorem does **not** establish that exact fixpoint detection is impossible for every incrementalization of a total computable query. It concerns arbitrary circuits in a Turing-complete instance of DBSP, which admits computations that need not converge on every input.

We now use “impossibility of exact fixpoint detection” instead of “undecidability” to make the runtime requirement clearer: a detector must never report convergence incorrectly and must eventually report it whenever it occurs. It need not decide nonconvergence or return a final yes/no answer on every input.

Precisely, if the choice of primitive nodes makes the DBSP circuit language Turing-complete, there exists a fixed level-1 circuit c with no computable sound and complete fixpoint detector. The assumption concerns the expressiveness of composed circuits, not individual primitive nodes. The proof implements a total primitive-recursive bounded evaluator as a circuit E and feeds it the program code and an increasing counter. On input $\delta_0(e)$, the resulting circuit c produces only zeros if the program encoded by e does not halt on input 0, and eventually only ones if it halts. Running a hypothetical exact computable detector until its first true output would therefore halt exactly for inputs encoding nonhalting programs. This would recognize the complement of the halting problem, which is not recursively enumerable. The core reduction is formalized in Lean 4.

The construction differs from the setting in your recomputation argument: it does not represent a total query with an independently computable final answer on every input. Its stream-elimination result is defined for nonhalting program codes but undefined for halting codes. Our theorem therefore does not rule out detectors that use a terminating non-incremental algorithm or other query-specific structure.

Moreover, knowing that an incremental computation equals the reference answer whenever it reaches a true fixpoint gives only one implication. Stopping when equality is observed additionally requires the converse: equality with the reference answer must imply that a true fixpoint has been reached. That need not hold for arbitrary DBSP circuits, since an intermediate result may transiently equal the final answer and later change again. When a particular query and incremental algorithm provide the converse, comparison with a non-incremental computation can indeed yield an exact detector.

We retain the theorem because it limits what can be expected for arbitrary circuits, while emphasizing that useful fragments can behave better. In particular, our later results show that IntConv is sound for all circuits and complete for regular circuits, including Datalog and while queries. We separately extend this completeness guarantee to their incrementally optimized forms, which need not themselves be regular circuits. Section 4.2 now states the theorem’s assumption, gives the circuit construction explicitly, and includes a footnote explaining the total-query limitation. The introduction, overview, and conclusion restrict the impossibility claim to arbitrary expressive circuits.

2. Role and evaluation of IntConv and the StFP detector

We agree that if convergence detection made incremental execution slower than non-incremental recomputation on realistic workloads, it would undermine its practical value. We have not implemented or benchmarked our global IntConv detector, so the paper makes no empirical efficiency claim. The detector may be fast, but our present results do not answer that question.

The contribution of IntConv here is primarily theoretical. It gives a declarative account of the internal-state-stability strategy used in practice. We prove that IntConv soundly implies external convergence, give a sound and complete structural detector for StFP, and use it to establish that IntConv is decidable. Our completeness results then identify broad circuit classes on which this implementable criterion coincides with the external mathematical specification. These are correctness and expressiveness guarantees rather than performance measurements.

The general StFP detector is therefore a correctness-oriented fallback, not a proposed optimal implementation. Implementations may optimize it or use specialized detectors justified by circuit structure. For example, FirstZero is already sound and complete for the naive and semi-naive query bodies.

Feldera already implements operator-level internal-state-stability checks for its supported programs. What remains unimplemented from our work is the general compositional detector across all bracketed sub-circuits, especially nested ones. The Feldera compiler version we inspected rejects nested recursive operators within recursive code, so evaluating this additional capability would require extending compiler and runtime support rather than merely enabling an existing check.

A meaningful performance evaluation also requires integration into a complete incremental-query system. Detector overhead and comparison with non-incremental recomputation depend on concrete state representations, data structures, scheduling, queries, workloads, update sizes, and other system optimizations abstracted away by our core calculus. An isolated implementation of the structural algorithm would show executability but would not answer the end-to-end performance question.

The revision now makes this scope explicit: Sections 2 and 5 describe the implementation argument as qualitative rather than empirical; Section 6 presents StFP as a general fallback and discusses specialized detectors; and the conclusion identifies implementation, detector-overhead measurement, and end-to-end comparison as future work.